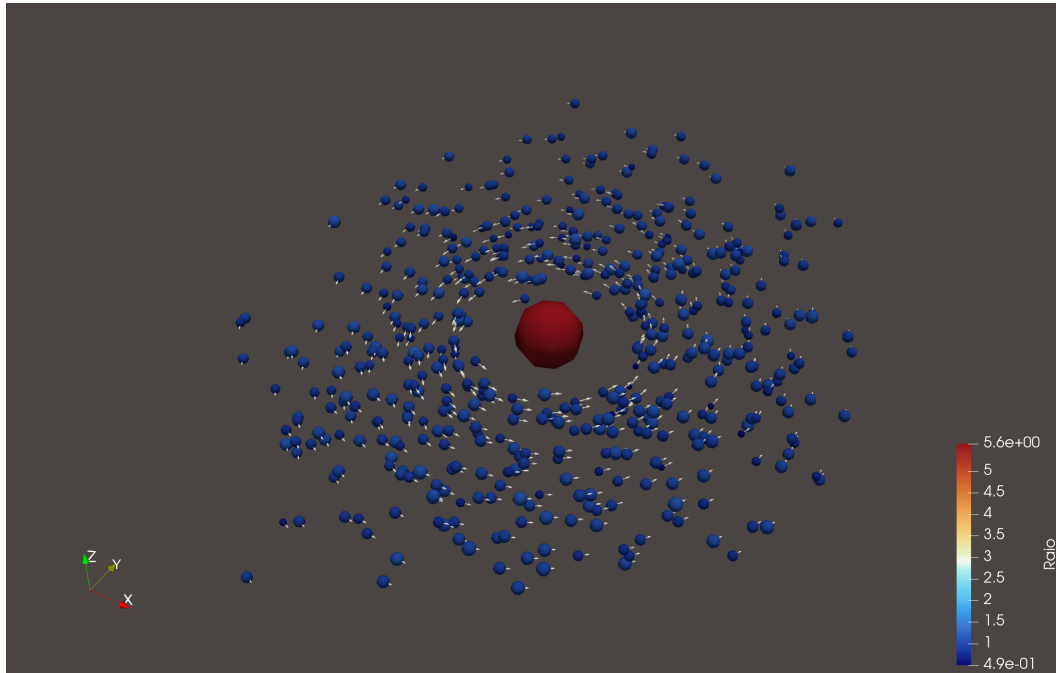




Challenge 01 – n-body simulation and visualization

Luís Paulo Santos, Dezembro, 2025

The goal of this challenge is for you to further develop the base code provided for simulating an n-body configuration subject to gravitational forces. Furthermore, you are required to develop an animation of the data using Paraview.

Clone the repository https://github.com/luisps/MCA_nBody.git

Initial Setup

Build the executable and run it.

This initial version will only simulate the initial conditions, i.e., no time steps.

You should see file `particles_00000000.vtk`

The constants in `constants.h` govern much of the simulation:

```
// gravitational constant
float const G = 6.674e-11 ;

// number of particles
int const N = 500;

// Sun mass
float const sunMass = 750.f;

// Delta t per iteration
float const DELTA_T = 200.f;

// number of simulation iterations
int const T_STEPS = 0;
```

The source file `n-body.cpp` contains the initialization of the data set, code for simulating time steps (each an interval of time given by `DELTA_T`), and code to delete the data set, respectively:

```
bool nBody_init (PARTICLE &dataSet);
bool nBody_tStep (PARTICLE &dataSet);
void nBody_close (PARTICLE dataSet);
```

The dataset consists of a list of particles. The datatype is defined in:

```
typedef struct {
    int N;                // number of particles
    float *Px, *Py, *Pz;  // arrays of particles' positions
    float *mass;          // array of particles' masses
    float *Vx, *Vy, *Vz;  // arrays of particles' velocities
} PARTICLE;
```

The file `VTK-Legacy.cpp` outputs the particles' data in an ASCII VTK legacy format that is recognized by Paraview.

For each particle, properties include its 3D Cartesian position, its mass, and its linear 3D instantaneous velocity. Note that this data consists of a cloud of particles; there is no grid associated with it.

Step 1 – Visualization

Design your own visualization of the data.

The image above was captured from Paraview by:

1. Loading the data

2. Calculating each particle radius using the calculator filter: $r = \sqrt[3]{\frac{3*mass}{4\pi}}$
3. Visualizing each particle as a sphere with the above radius using the Glyph filter
4. Visualizing each particle's velocity as an arrow using a Glyph filter directly connected to the data source

Feel free to improve and/or use completely different visual metaphors.

Step 2 – Time series simulation

By changing the T_STEPS constant (e.g., 500), you can enable simulation of multiple time steps, each corresponding to piecewise constant velocity and acceleration approximations during the time interval given by DELTA_T.

The current simulation algorithm is:

```
// for each particle
for (int i=0 ; i < N ; i++) {
    // compute acceleration due to all other particles
    float_V a=0;
    for (int j=0 ; j < N ; j++) {
        // compute the direction vector
        float_V dir = dataSet.P[j] - dataSet.P[i];

        float const dist_squared =
            (i==j ? 1.f : dirX*dirX + dirY*dirY + dirZ*dirZ) ;

        float const dist = sqrtf(dist_squared);

        // normalize direction vector
        dir /= dist;

        float const acc = (i==j ? 0.f : G ) * dataSet.mass[j] /
dist_squared;

        // add the acceleration due to j onto i
        a += acc * dir;
    }

    // update particle i position
    dataSet.P[i] = dataSet.P[i] + DELTA_T * dataSet.V[i] + 0.5f
* a * DELTA_T * DELTA_T;

    // update particle i velocity
    dataSet.V[i] += a * DELTA_T;
}
```

Build the executable, design your visualization and examine your animation.

Step 3 – Debugging the simulation

You certainly noticed weird behaviors. Improve the simulations so that the particles' behavior is as expected. Note that whether they diverge to space, orbit, or crash into the Sun will depend on the initial conditions.

A problem with the current algorithm is that it allows particles to become infinitely close to each other without either lower bounding the distance or detecting and processing collisions. Note that very small distances will result in exponentially large velocities.

A suggestion is that before each simulation, the particles are tested for proximity, and if their distance is smaller than the sum of their radii, they are merged. Merging implies summing the masses and weight-averaging positions and velocities (with the individual masses acting as the weights). Remember that this may require several passes, as clusters of multiple closely packed particles may form.

Step 4 – Scaling the simulation

Use your advanced computing skills to optimize the performance of the simulation code, such that large numbers (millions ?) of particles can be handled.

Does the code vectorize?

Can thread level parallelism be exploited?

Can multiple computing nodes be used?

Step 5 – Smarter n-body algorithms

The current simulation algorithm evaluates all N^2 particle interactions. This is infeasible for large datasets. Can you propose more efficient algorithms, even if less accurate?

Can the data be spatially sorted, and nearby particles be treated with more accuracy than farther ones?

Can a Monte Carlo simulation be used, thus sampling only the attraction of a subset of the particles?